
Adaptive Action Chunking via Multi-Chunk Q Value Estimation

Yongjae Shin Jongseong Chae Seongmin Kim Jongeui Park Youngchul Sung
KAIST
{yongjae.shin, ycsung}@kaist.ac.kr

Abstract

Action chunking emerged as a pivotal technique in imitation learning, enabling policies to predict cohesive action sequences rather than single actions. Recently, this approach has expanded to reinforcement learning (RL), enhancing behavioral consistency and reducing bootstrapping errors in value function estimation. However, existing methods rely on a fixed chunk length, creating a performance bottleneck as the optimal length varies across states and tasks. In this paper, we propose Adaptive Action **CH**unking (ACH), a novel offline-to-online RL algorithm that dynamically modulates chunk length during both training and inference. To find the optimal chunk length for a dynamically varying current state, we simultaneously estimate action-values for all candidate chunk lengths in a single forward pass, using a Transformer-based architecture. Our mechanism allows the agent to select the most effective chunk length adaptively based on the current state. Evaluated on 34 challenging tasks, ACH consistently outperforms fixed-length baselines, demonstrating superior generalization and learning efficiency in complex environments.

1 Introduction

Action chunking [48] is a method introduced in imitation learning (IL), in which a policy is trained to predict a sequence of actions simultaneously, rather than predicting only a single action at each step. By generating an action sequence for a given state and executing it in an open-loop manner, action chunking enables more efficient inference. Moreover, unlike single-action prediction, which may lead to redundant or inconsistent behaviors as each action is sampled individually, an action-chunking policy promotes behavioral consistency by producing a cohesive sequence. Driven by these advantages, action chunking has emerged as a pivotal technique within the imitation domain [48, 3, 4, 12, 14, 7].

Recently, action chunking has expanded beyond IL into the realm of reinforcement learning (RL) [39]. Specifically, in offline RL [19], which shares similarities with IL due to its reliance on static and pre-collected datasets, several methods have been proposed by successfully integrating action chunking [38, 34, 13, 21, 29, 22]. These approaches focus on effective value function training under the assumption of policies with fixed chunk lengths. By aligning value estimation with action sequences rather than individual actions, these methods effectively mitigate the bootstrapping errors inherent in single-action estimation. Consequently, they demonstrate the advantage of action chunking in offline RL, particularly in long-horizon tasks.

Furthermore, action chunking has been incorporated into offline-to-online (Off2On) RL, a framework introduced to overcome the fundamental limitation of offline RL that the policy performance is inherently constrained by the quality of given datasets [18, 46, 28, 42, 22, 35]. In Off2On RL, a policy is pre-trained via offline RL and then used as an initialization for subsequent online training. This transition allows the agent to collect additional data through its own actions by interacting

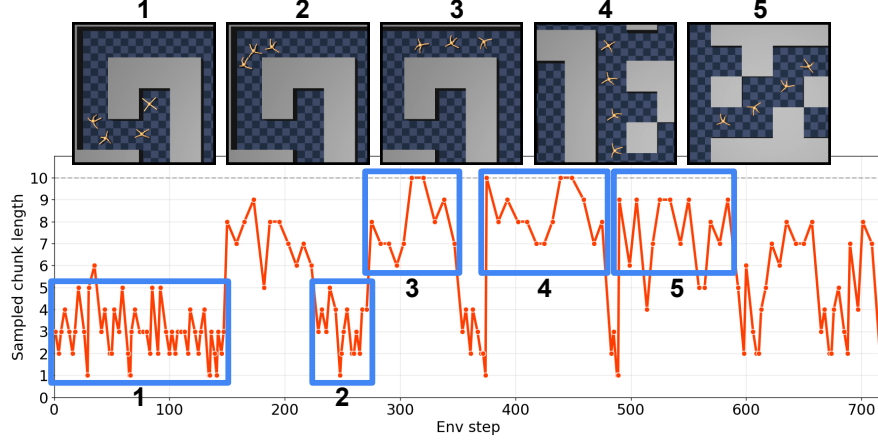


Figure 1: Adaptive Action Chunking: Optimal chunk length over environment timestep during an antmaze-giant episode. The number of blue squares corresponds to the subfigures above. The optimal chunk length for each state is computed by our proposed algorithm, ACH.

with the environment, thereby refining its policy to surpass the limitations of the given dataset and improve performance. Thus, the behavioral consistency of action chunking provides an additional advantage in Off2On RL by enabling efficient exploration, on top of the aforementioned improved value function learning [22].

Despite the chunk length being a critical hyperparameter that significantly influences overall performance in action chunking, existing studies in offline and Off2On RL have employed fixed chunk lengths for both training and inference. However, a static chunk length is often suboptimal across different tasks, and even within a single task, its optimality fluctuates over time. Consider, for example, the antmaze navigation task [30], where an ant must traverse a complex, large-scale maze to reach a goal, as illustrated in Figure 1. When the ant is in the segment shown in Subfigure 1, the state evolves rapidly; here, an excessively large chunk length may lead to frequent collisions or cause the agent to overshoot turning points, thereby degrading performance. In such scenarios, a shorter chunk length is preferred. Conversely, in the segment shown in Subfigure 3, where the trajectory is more predictable, the agent should ideally maintain its momentum to reach the goal efficiently. In this case, a larger chunk length is more advantageous. These variations underscore the need to move beyond rigid, fixed-length approaches toward a more adaptive action chunking strategy.

In this paper, we propose **Adaptive Action CHunking (ACH)**, a novel Off2On RL algorithm designed to address the aforementioned limitations. Our method enables the policy to optimally adjust its chunk length based on the current state during both environmental interaction and inference, as illustrated in Figure 1. To achieve this, we leverage a Transformer-based value function capable of simultaneously estimating the values of all possible chunk lengths in a single forward pass. By leveraging this architecture, ACH dynamically selects the most suitable chunk length according to the computed action-values, ensuring both effective and efficient learning. We evaluate our approach on 34 diverse and challenging tasks, where experimental results demonstrate that ACH successfully modulates chunk length across various environments, consistently achieving state-of-the-art performance in Off2On RL.

2 Preliminaries

Offline-to-online RL In this paper, we consider a Markov Decision Process (MDP) [39] defined as $\mathcal{M} = (\mathcal{S}, \mathcal{A}, r, \rho, \mathcal{P}, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $r(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ represents the reward function, $\rho \in \Delta(\mathcal{S})$ is the initial state distribution, $\mathcal{P}(s'|s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the transition kernel, and $\gamma \in [0, 1]$ is the discount factor. The objective of RL is to find a policy $\pi(a|s)$ that maximizes the expected return: $\mathbb{E}_{\pi} [\sum_k \gamma^k r(s_k, a_k)]$. Off2On RL is a two-stage learning framework that performs offline pre-training and subsequent online learning [45, 28, 25, 42, 18, 36, 46, 22, 47, 49, 8]. In the offline pre-training stage, the agent is trained on a pre-collected dataset $D = \{(s, a, r, s')\}$ without any interaction with the environment. During the

subsequent online learning stage, the trained policy interacts directly with the environment, further refining its behavior.

Action Chunking in RL Action Chunking, originally introduced in IL to improve robotic control efficiency [48], has become a key technique in robotics research [7, 3, 4, 12, 14]. By grouping a sequence of actions into a single unit, referred to as a chunk, this approach reduces decision-making frequency while promoting consistent execution. Recently, the chunking approach has extended into RL [44, 22, 21, 20, 41, 29, 13, 34, 38, 44]. Instead of learning the value of a single action a_t as in conventional approaches, the value of an action chunk $\mathbf{a}_{t:t+h} := (a_t, a_{t+1}, \dots, a_{t+h})$ is learned to enable faster value propagation and unbiased value estimation, thereby facilitating effective and stable temporal-difference (TD) learning [22, 21, 13, 38].

In the context of Off2On RL, action chunking can be further leveraged to enhance online exploration through consistent behavior. Q-Chunking (QC) [22] integrates this benefit with the aforementioned advantages in value learning, employing a flow-based policy as its backbone. To this end, a flow-based behavior policy β_ψ is trained by learning a velocity field v_ψ in a chunk-wise manner:

$$\mathcal{L}_\beta(\psi) = \mathbb{E}_{\substack{(s_t, \mathbf{a}_{t:t+h}) \sim D, \\ u \sim \text{Unif}([0,1]), \\ \mathbf{z} \sim \mathcal{N}(0, I_{A(h+1)})}} \left[\|v_\psi(s_t, \mathbf{a}_u, u) - (\mathbf{a}_{t:t+h} - \mathbf{z})\|_2^2 \right] \quad (1)$$

where $\mathbf{a}_u = (1-u)\mathbf{z} + u\mathbf{a}_{t:t+h}$ and A denotes the action space dimension. Unlike conventional single-action learning, the velocity field here is defined over action chunks. By integrating the trained velocity field v_ψ from initial noise $\mathbf{z} \sim \mathcal{N}(0, I_{A(h+1)})$, the behavior policy samples an action chunk $\mathbf{a}_{t:t+h}$ for a given state s_t . Subsequently, behavior-regularized Q-maximization is applied to derive a one-step flow policy [32]:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{\substack{s_t \sim D, \\ \mathbf{z} \sim \mathcal{N}(0, I_{A(h+1)})}} \left[-Q_\phi(s_t, \pi_\theta(s_t, \mathbf{z})) + \alpha \| \pi_\theta(s_t, \mathbf{z}) - \beta_\psi(s_t, \mathbf{z}) \|_2^2 \right] \quad (2)$$

where α is a hyperparameter controlling the strength of the regularization. Beyond policy action generation, the action-value function is also learned in a chunk-wise manner via temporal difference (TD) learning:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{\substack{(s_t, \mathbf{a}_{t:t+h}, \mathbf{r}_{t:t+h}, s_{t+h+1}) \sim D, \\ \mathbf{a}'_{t+h+1:t+2h+1} \sim \pi_\theta(\cdot | s_{t+h+1})}} \left[(Q_\phi(s_t, \mathbf{a}_{t:t+h}) - (R_{t:t+h} + \gamma^h Q_{\bar{\phi}}(s_{t+h+1}, \mathbf{a}'_{t+h+1:t+2h+1})))^2 \right] \quad (3)$$

where $R_{t:t+h} = \sum_{k=0}^h \gamma^k r_{t+k}$ and $\bar{\phi}$ denotes the target network parameters. Through this framework, both the generation and evaluation of actions are unified into a chunk-wise operational paradigm.

3 Motivation

Consider the problem of determining the optimal chunk length h for a given state s_t . To formulate this mathematically, we must establish an optimality criterion that accounts for the underlying factors influencing performance. The first factor is **the uncertainty of predicted future states**. In action chunking, a sequence $\mathbf{a}_{t:t+h} = (a_t, a_{t+1}, \dots, a_{t+h})$ is sampled by a policy π conditioned solely on s_t and executed in an open-loop manner. If the actual trajectory deviates significantly from the states implicitly predicted by the policy at time t , the resulting actions become suboptimal, leading to performance degradation. Under this criterion, a chunk length of one would be ideal to remain responsive to rapid state changes (left of Figure 2, corresponding to Subfigure 1 in Figure 1). However, such an extreme setting eliminates the benefit of chunking in terms of action-generation complexity. One could instead determine the maximum h within an allowable uncertainty threshold $H_{threshold}$, using the estimated cumulative entropy of the induced Markov chain:

$$\hat{H}(s_{t+1}, s_{t+2}, \dots, s_{t+h} | s_t) = \sum_{i=t+1}^{t+h} \hat{H}(s_i | s_{i-1}). \quad (4)$$

A recent study by Liang et al. [23] falls into this category, although they utilize the average action entropy and its maximum gap as a heuristic for chunk length selection.

A crucial insight of our work is that a chunk length of one is not always optimal, even when computational complexity is disregarded (middle of Figure 2, corresponding to Subfigure 3 in Figure 1). Suppose future states are highly correlated with s_t , i.e., $s_{t+n} \approx s_t, \forall n \leq h$. In this case, an action

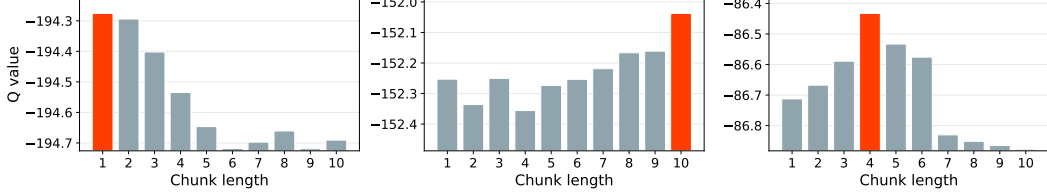


Figure 2: Action value as a function of chunk length (antmaze-giant-navigate with maximum chunk length set as 10): (left) fast-changing states, (middle) strongly-correlated states over time, and (right) trade-off situation. The minimal value difference stems from the reward design of the environment: a constant reward of -1 per step, complemented by a sparse reward (1 or 0).

sequence sampled at s_t remains highly relevant throughout the horizon h . If we use a chunk length of one, each newly sampled action interrupts the continuity of behaviors that a larger chunk would provide [22]. This issue is particularly pronounced in stochastic policies, such as flow-based models. In these policies, each independent sampling of a_t introduces a random component that manifests as jitter, hindering the agent’s progress toward the goal. We refer to this performance degradation as **policy stochasticity jittering loss**. Consequently, from a pure performance standpoint, the optimal chunk length is determined by the trade-off between future state uncertainty and policy stochasticity jittering, as illustrated in Figure 2 (right), corresponding to Subfigure 5 in Figure 1.

4 Method

In this section, we present ACH, a novel framework designed for Off2On RL. The proposed method consists of three fundamental components:

Optimality via Chunk Values: To determine the most appropriate chunk length for a given state, we utilize the action-values of all candidate chunk lengths as the primary selection criterion.

Transformer-based Value Estimation: We employ a Transformer-based value function to efficiently estimate these values for multiple lengths in parallel.

Adaptive Sampling Mechanism: Based on the learned value function, we introduce an adaptive sampling mechanism that dynamically adjusts the chunk length in response to the policy’s current state.

By integrating these components, ACH achieves an optimal trade-off between future state uncertainty and policy stochasticity jittering, yielding superior performance.

4.1 Value-based Optimality

To balance the trade-off between future state uncertainty and policy stochasticity jittering, one might consider a formulation using predicted state entropy (4) alongside an action jittering loss, defined as $\sum_{k=t}^{t+h-1} f(\|a_{k+1} - a_k\|^2)$ with a monotonic function $f(\cdot)$. An overall loss could then be constructed as $\hat{H}(s_{t+1}, s_{t+2}, \dots, s_{t+h} | s_t) + \lambda \sum_{k=t}^{t+h-1} f(\|a_{k+1} - a_k\|^2)$ to determine the optimal h . However, a primary drawback of this approach is that the resulting h may not yield optimal performance, as both state uncertainty and jittering losses influence performance only indirectly. Furthermore, such a framework necessitates the design of multiple heuristic functions and extensive hyperparameter tuning. To circumvent these challenges, we directly employ the values of action chunks as our optimality criterion, aligning with the fundamental goal of RL: return maximization. To this end, we construct a set $\mathcal{A}_h$ of action chunks with lengths up to h , all originating from s_t , as follows:

$$\mathcal{A}_h = \{(s_t, a_t), (s_t, a_t, a_{t+1}), \dots, (s_t, a_t, a_{t+1}, \dots, a_{t+h})\}, \quad (5)$$

where the n -th element e_n of this set includes the $(n-1)$ -th element e_{n-1} as its prefix, i.e., $e_n = (e_{n-1}, a_n)$. Suppose that we have a sufficiently-trained policy π . Then, consider the value of an action chunk $(a_t, \dots, a_{t+n})$ for given s_t under π , defined as

$$Q^\pi(s, a_t, \dots, a_{t+n}) = \mathbb{E}_{a_{t+n+1}, a_{t+n+2}, \dots \sim \pi} \left[\sum_{k \geq 0} \gamma^k r_{t+k} | s_t, a_t, \dots, a_{t+n} \right] \quad (6)$$

and consider the value sequence for all chunks in $\mathcal{A}_h$ from (5):

$$Q^\pi(s_t, a_t), Q^\pi(s_t, a_t, a_{t+1}), \dots, Q^\pi(s_t, a_t, a_{t+1}, \dots, a_{t+h}). \quad (7)$$

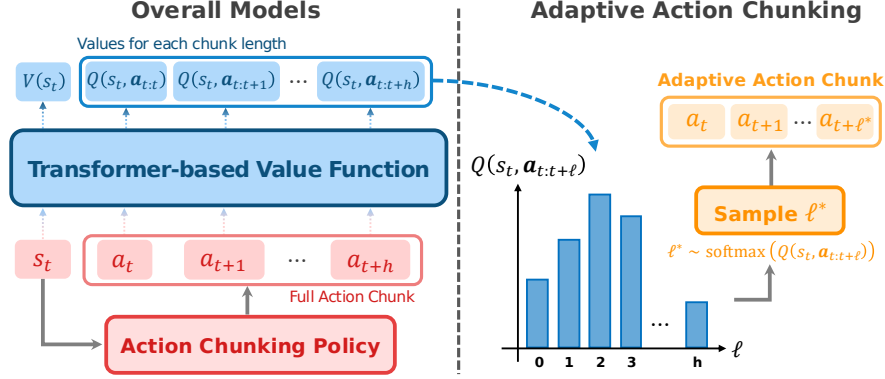


Figure 3: The overall architecture of ACH: (left) causal Transformer-based joint value learning and (right) chunk length sampling

This sequence progressively evaluates the value of action chunks $(a_t, \dots, a_{t+n})$ for given s_t , with n ranging up to h . When future states are highly predictable, longer chunks yield higher values, thereby mitigating policy stochasticity jittering. Conversely, if predictability is low, longer chunks result in lower values because actions planned at s_t deviate from the actual future states. The key of this approach lies in the prefix structure of the chunk sequence. Specifically, if we were to compare $Q^\pi(s_t, a_t, a_{t+1})$ and $Q^\pi(s_t, a'_t, a'_{t+1}, a_{t+2})$ where $(a_t, a_{t+1}) \neq (a'_t, a'_{t+1})$, the difference in Q-values could stem from the initial action mismatch rather than the chunk length itself. By maintaining a prefix-consistent sequence, we can isolate the effect of length and determine the optimal chunk length within the range $[1, h + 1]$ by selecting the index that maximizes the value in the sequence in (7).

4.2 Transformer-based Multi-Chunk Value Estimation

The main hurdle to realizing our adaptive chunk length selection described above is how to learn the Q values of multiple increasing chunks in (7) simultaneously. To solve this problem, we adopt a *causal Transformer* as our value function architecture. Building on previous work [20], which utilized a Transformer-based value function for episodic RL, we design a value function that takes $(s_t, \mathbf{a}_{t:t+h})$ as input and outputs $(V(s_t), Q(s_t, \mathbf{a}_{t:t}), Q(s_t, \mathbf{a}_{t:t+1}), \dots, Q(s_t, \mathbf{a}_{t:t+h}))$, as illustrated in Figure 3, where the state value function $V(s)$ is additionally included for efficient bootstrapping. With a causal Transformer as the value function, we can estimate the values of all possible action chunks in a single forward pass, while ensuring that action chunks do not attend to future steps.

For efficient training under multiple chunk lengths, we consider the following learning objective that trains simultaneously over different chunk lengths while enabling accurate value estimation for the current policy:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{\substack{(s_{t:t+h}, \mathbf{a}_{t:t+h}, \\ \mathbf{r}_{t:t+h}, s_{t+h+1}) \sim D, \\ \mathbf{a}'_{t:t+h} \sim \pi_\theta(\cdot | s_t)}} \left[\frac{1}{h+1} \sum_{n=0}^h \left(Q_\phi(s_t, \mathbf{a}_{t:t+n}) - (R_t^n + \gamma^{n+1} V_{\bar{\phi}}(s_{t+n+1})) \right)^2 \right. \\ \left. + \left(V_\phi(s_t) - \mathbb{E}_{\ell \sim \pi_L(\cdot | s_t, \mathbf{a}'_{t:t+h})} [Q_{\bar{\phi}}(s_t, \mathbf{a}'_{t:t+\ell})] \right)^2 \right] \quad (8)$$

where $R_t^n = \sum_{k=0}^n \gamma^k r_{t+k}$, $\mathbf{a}'_{t:t+\ell}$ is a truncation (up to ℓ) of full chunk $\mathbf{a}'_{t:t+h}$ sampled from $\pi_\theta(\cdot | s_t)$, and $\pi_L(\ell | s_t, \mathbf{a}'_{t:t+h})$ denotes the chunk length selection policy over $\ell \in \{0, 1, \dots, h\}$, which will be explained shortly in Section 4.3.

The loss above consists of two terms. The first term learns the action value for each chunk length n , using the n -step target. By estimating the value of the action chunk $\mathbf{a}_{t:t+n}$ covering the n -step reward $\mathbf{r}_{t:t+n}$, the proposed method yields less unbiased value estimation. Moreover, unlike the previous approach [22] that requires the state-action value of the next n -step chunk $\mathbf{a}_{t+n+1:t+2n+1}$ when applying bootstrapping, our method instead builds the target from the value of the state s_{t+n+1} , thereby avoiding the variance from action-based bootstrapping. The second term learns the state

value from the current policy’s behavior. In doing so, it performs value estimation according to the chunking length selection distribution. Consequently, our method provides less biased and more accurate value estimates for the current policy. Note that the result in Figure 2 is obtained by this transformer-based value function.

Based on the learned value function, we train the policy π_θ by jointly optimizing distillation from the behavior policy β_ψ learned via flow matching and action-value maximization:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{\substack{s_t \sim D, \\ \mathbf{z} \sim \mathcal{N}(0, I_{A(h+1)}), \\ \mathbf{a}'_{t:t+h} \sim \pi_\theta(s_t, \mathbf{z})}} \left[-\mathbb{E}_{\ell \sim \pi_L(\cdot | s_t, \mathbf{a}'_{t:t+h})} [Q_\phi(s_t, \mathbf{a}'_{t:t+\ell})] + \alpha \|\mathbf{a}'_{t:t+h} - \beta_\psi(s_t, \mathbf{z})\|_2^2 \right]. \quad (9)$$

where α is a behavior cloning coefficient. This training process also includes the chunk length selection policy π_L , guiding the policy to learn action chunks with high action values.

4.3 Adaptive Chunk Length Sampling

We generate a full action chunk $\mathbf{a}'_{t:t+h}$ for current state s_t using the policy $\pi_\theta(\cdot | s_t)$, and feed the full sequence $(s_t, \mathbf{a}'_{t:t+h})$ to the transformer-based value function. This yields the Q values of all $h + 1$ prefix chunks: $(s_t, \mathbf{a}'_{t:t}), (s_t, \mathbf{a}'_{t:t+1}), \dots, (s_t, \mathbf{a}'_{t:t+h})$. One could then select the optimal chunk length for s_t as $h^* = 1 + \arg \max_{n \in \{0, 1, \dots, h\}} Q_\phi(s_t, \mathbf{a}'_{t:t+n})$, resulting in a greedy selection policy. However, inspired by ϵ -greedy strategies in Q-learning, we further refine this selection process to incorporate exploration capabilities during online learning. Consequently, we define the chunk length selection policy, π_L , as a soft policy following a sampling distribution:

$$\pi_L(\ell) = \frac{\exp(\beta \cdot Q_\phi(s_t, \mathbf{a}'_{t:t+\ell}))}{\sum_{n=0}^h \exp(\beta \cdot Q_\phi(s_t, \mathbf{a}'_{t:t+n}))}, \quad \forall \ell \in \{0, 1, \dots, h\} \quad (10)$$

where β denotes the inverse temperature parameter. Since both the policy and the value function are trained to prioritize chunks with high action values in the offline dataset via Eqs. (8) and (9), this method naturally selects action chunks that align with the training dynamics. This probabilistic sampling encourages the agent to explore diverse chunk lengths, thereby enabling it to identify the most effective one.

4.4 Practical Implementation

We adopt QC [22] as our backbone model. QC offers two distinct variants: one that solely trains and samples from a behavior policy, and another that trains a one-step policy in addition to the behavior policy, similar to FQL [32]. To ensure inference efficiency, we employ the latter variant. Our Transformer model, designed to estimate values for multiple chunk lengths, incorporates causal masking to strictly prevent attending to future information. Furthermore, as our proposed method adaptively modulates chunk lengths, we extend the maximum horizon to 10 ($h = 9$) in most environments, doubling the default horizon used in the original QC framework. Although the temperature β can be adjusted to encourage exploration in the early online learning and prioritize exploitation in later stages, we find that fixing $\beta = 1$ is empirically sufficient and adopt this setting. Additional implementation details, hyperparameter configurations, and algorithm pseudo-code are provided in Appendix B.4.

5 Experiments

In this section, we empirically evaluate the proposed method, ACH, across challenging environments involving long-horizon and diverse tasks in comparison with several key baselines.

Benchmarks and Tasks We primarily evaluate ACH on OGBench [30], which encompasses a variety of challenging tasks. Although OGBench was originally designed for goal-conditioned RL, we adopt its single-task variants to align with our setting. We conduct experiments across six environments in OGBench: antmaze-giant, humanoidmaze-medium, antsoccer-arena, cube-triple/quadruple, puzzle-4x4. Each environment consists of five tasks. For each environment, we use the default dataset, except for cube-quadruple, where a dataset of 10M transitions is used. Additionally, we evaluate on four tasks from Robomimic [26]: lift, can, square, transport. For all robomimic environments, we utilize the multi-human dataset.

Table 1: Aggregated results of ACH and baselines on OGBench. All methods are evaluated after 1M steps of offline learning followed by 1M steps of online learning. We report the mean and 95% confidence intervals across 10 seeds. Results for each environment are averaged over five tasks.

Task	RLPD	BFN	FQL	FINO	QC	ACH
antmaze-giant-navigate	$0 \pm 0 \rightarrow 69 \pm 9$	$0 \pm 0 \rightarrow 22 \pm 6$	$10 \pm 5 \rightarrow \mathbf{74} \pm 7$	$2 \pm 2 \rightarrow \mathbf{76} \pm 4$	$0 \pm 0 \rightarrow 47 \pm 8$	$1 \pm 1 \rightarrow \mathbf{72} \pm 2$
antsoccer-arena-navigate	$0 \pm 0 \rightarrow 64 \pm 4$	$51 \pm 2 \rightarrow 74 \pm 7$	$35 \pm 2 \rightarrow 70 \pm 5$	$40 \pm 3 \rightarrow 71 \pm 5$	$10 \pm 2 \rightarrow 67 \pm 2$	$0 \pm 0 \rightarrow \mathbf{84} \pm 1$
cube-quadruple-play-10M	$0 \pm 0 \rightarrow 0 \pm 0$	$2 \pm 1 \rightarrow 10 \pm 2$	$0 \pm 0 \rightarrow 2 \pm 1$	$1 \pm 0 \rightarrow 7 \pm 3$	$1 \pm 1 \rightarrow 33 \pm 3$	$0 \pm 0 \rightarrow \mathbf{52} \pm 6$
cube-triple-play	$0 \pm 0 \rightarrow 0 \pm 0$	$4 \pm 1 \rightarrow 22 \pm 3$	$2 \pm 1 \rightarrow 19 \pm 2$	$2 \pm 1 \rightarrow 24 \pm 3$	$4 \pm 2 \rightarrow 57 \pm 4$	$1 \pm 1 \rightarrow \mathbf{67} \pm 3$
humanoidmaze-medium-navigate	$0 \pm 0 \rightarrow 29 \pm 9$	$59 \pm 3 \rightarrow \mathbf{87} \pm 6$	$36 \pm 3 \rightarrow 56 \pm 3$	$38 \pm 5 \rightarrow \mathbf{92} \pm 4$	$15 \pm 2 \rightarrow 32 \pm 4$	$45 \pm 2 \rightarrow 82 \pm 1$
puzzle-4x4-play	$0 \pm 0 \rightarrow 64 \pm 12$	$32 \pm 2 \rightarrow 60 \pm 7$	$14 \pm 2 \rightarrow 54 \pm 6$	$15 \pm 3 \rightarrow 53 \pm 7$	$11 \pm 2 \rightarrow 38 \pm 4$	$13 \pm 3 \rightarrow \mathbf{90} \pm 8$

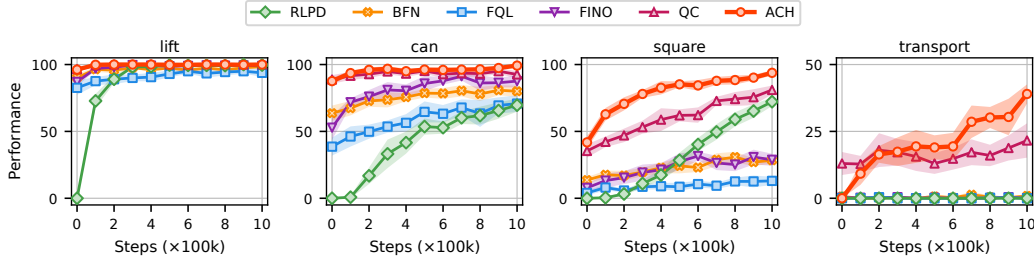


Figure 4: Learning curves over 1M online steps across robomimic tasks. The performance is averaged over 10 random seeds, with the 95% confidence interval as the shaded region.

Baselines We compare ACH against the following Off2On RL methods: (1) RLPD [2], an online RL algorithm that starts with an offline dataset and achieves high performance without an offline learning phase. (2) BFN is a method introduced in the QC paper [22], which draws multiple action candidates from a behavior policy and selects the one with the highest estimated action value. (3) FQL [32], an offline RL method that learns a behavior policy via flow matching and distills it into a one-step policy, demonstrating strong performance across offline and Off2On RL. (4) FINO [35], an algorithm targeting Off2On RL with a flow-based policy, which builds upon FQL and injects noise into the flow matching for strong Off2On performance. (5) QC [22], as discussed in Section 2, extends the conventional single-step action approach to action chunks, and serves as our backbone algorithm.

Evaluations For all baselines, we apply the same protocol to obtain experimental results. Training consists of 1M steps for offline learning followed by 1M steps for online learning, and the final results are based on the performance evaluated at the last step. All experiments are conducted with 10 random seeds per task, and we report the mean and 95% confidence intervals. To fairly compare the performance improvement during online learning, we also report the offline performance. We bold all results within 95% of the top-performing algorithm after online learning.

Results Table 1 summarizes the performance across 30 OGBench environments, aggregated by environment. The results demonstrate that ACH achieves superior performance across diverse domains. Specifically, RLPD, which conducts online RL without a prior offline phase, exhibits clear limitations inherent in its learning paradigm. Similarly, the performance of FQL suggests that simply extending offline RL methods to online fine-tuning is insufficient for addressing the challenges of the Off2On transition. BFN employs a sampling strategy similar to the proposed method, yet fails to achieve strong performance due to the inherent limitation of single-action prediction. While FINO performs competitively in navigation-heavy tasks such as *antmaze* and *humanoidmaze*, its performance in manipulation tasks (e.g., *cube*) reveals the limitations of single-action prediction in scenarios requiring precise manipulation. Lastly, while QC demonstrates that action chunking yields competitive results in most tasks, its comparison with ACH underscores that our proposed method facilitates more efficient learning. This gain is primarily attributed to ACH’s ability to employ adaptive chunk lengths, as opposed to the rigid, fixed-length approach of the baseline.

Figure 4 illustrates the learning curves for four environments within robomimic during the online fine-tuning. The results demonstrate that ACH consistently achieves the highest performance across all tasks. This performance gain is particularly pronounced in the *square* and *transport* tasks, where the backbone algorithm—limited by a fixed chunk length—exhibits marginal improvements. These findings, together with the results in Table 1, validate ACH as a robust framework that outperforms conventional methods by adaptively modulating chunk lengths across challenging domains.

5.1 Further Analysis

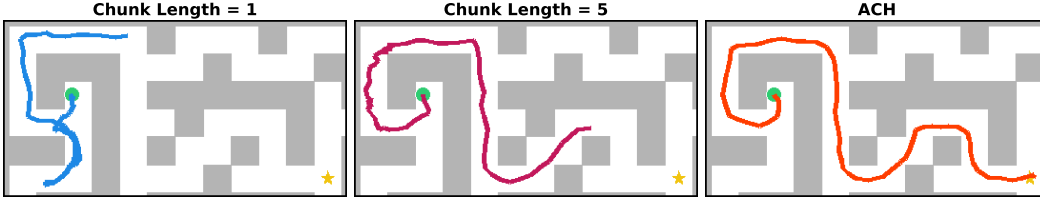


Figure 5: Behavioral comparison of ACH and fixed-length baselines in the `antmaze-giant` environment. Starting from a shared initial state, all algorithms execute the same number of total action steps. Agent trajectories are constructed by connecting sequential positions at each timestep. The initial state and goal state are marked by a green circle and a yellow star, respectively.

Behavior Analysis To verify the efficacy of the adaptive chunk length selection, we conducted a comparative analysis in the challenging `antmaze-giant` environment. In Figure 5, we compare the trajectories of our proposed method against two baselines: a model that does not utilize action chunking (employing a fixed chunk length of 1) and a backbone model using a static, fixed chunk length of 5. All baselines are trained within the same learning timestep, and the trajectories are recorded by sampling from the final trained policies.

The results show that the model without action chunking exhibits redundant, oscillatory (i.e., jittering) behaviors near the initial state. This is because the policy predicts only a single action at each step, leading to frequent inconsistencies in decision-making and highlighting the necessity of action chunking. While the fixed-length action chunking demonstrates improved behavior compared to single action prediction, it still suffers from suboptimal efficiency. For instance, it fails to execute efficient maneuvers in areas requiring precise control, such as navigating corners, and ultimately fails to reach the goal within the timestep limit. In contrast, our method selects the most appropriate chunk length for every state, resulting in efficient and smooth trajectories that successfully reach the goal. These behavioral patterns are accumulated as high-quality transitions during online learning, which facilitates sample-efficient training and enhanced performance. We present the results on the full map in Figure 5 and additional behavioral patterns discovered for chunk length 5 in Appendix D.2.

Ablation Study Section 4 presents the rationale for the value-based chunk length selection criterion (Section 4.1), describes the training of the value function and policy (Section 4.2), and details the adaptive chunk length sampling procedure (Section 4.3). To assess the impact of these core components, we conduct an ablation study by replacing each with an alternative approach. For the chunk length selection criterion, we examine a variant that randomly selects a length among all candidates instead of value-based sampling (denoted as `Random`). Regarding the training procedure, we evaluate a variant that assigns a uniform weight to all chunk length candidates, i.e., $\ell \sim \text{Unif}[1, h + 1]$ in Eqs. (8) and (9) rather than prioritizing the distribution π_L within the training losses (denoted as `Uniform`). To evaluate the sampling mechanism, we replace our soft sampling strategy Eq. (10) with a greedy approach that selects the chunk length with the highest estimated value (denoted as `Greedy`).

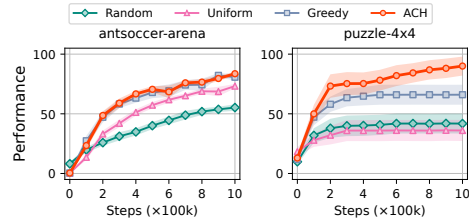


Figure 6: Experimental comparison of ACH against alternative training and sampling strategies. Each plot presents the aggregated performance across five distinct tasks.

As shown in Figure 6, replacing any of these components leads to a performance decline, confirming that no substitution matches the effectiveness of our proposed method. Simply selecting the chunk length at random does not consider the expected return of executing the corresponding action chunk, resulting in inferior performance across all environments. Training uniformly across all chunk lengths results in significant degradation, which we attribute to the misalignment between the training objective and the action selection policy used during online learning. The greedy alternative achieves competitive performance in some environments due to its behavior similarity to our method; however, since it solely exploits the Q-values, it fails to explore sufficiently. Consequently, it exhibits suboptimal sample efficiency compared to ACH. These findings confirm that the synergy between our training and sampling mechanisms is essential to enabling sample-efficient learning.

Evolution of Chunk Length In Off2On RL, the primary role of online learning is to enable the agent to refine its behavior through direct environment interaction. While methods with a fixed chunk length are limited to adjusting individual actions, our approach facilitates both behavioral correction and chunk length optimization. To investigate this, we analyze the evolution of the average chunk length during online learning. As illustrated in Figure 7, ACH exhibits a clear tendency to gradually increase the average chunk length as training progresses. This trend is attributed to the stitching of trajectories within the value function. As the policy optimizes based on this value function, it becomes capable of executing behaviors as single, extended sequences that previously required multiple short segments. These results indicate that our approach goes beyond merely refining individual actions; by leveraging value-based trajectory stitching to progressively increase chunk lengths, it ultimately facilitates sample-efficient learning.

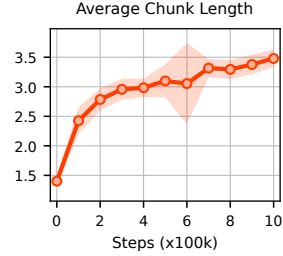


Figure 7: Evolution of average chunk length during the online learning phase on puzzle-4x4.

6 Related Work

Offline-to-online RL Off2On RL is a framework that addresses the inability of offline RL to learn beyond the given dataset and the safety and cost issues of applying online RL from scratch [18, 46, 28, 45, 25, 42, 35, 36, 47, 49]. The objective is to leverage the offline dataset to prepare a policy, then refine it through direct environment interaction. The most straightforward approach is to extend existing offline RL methods [17, 16, 40, 43] to online learning, but such approaches fail to fully exploit the interaction due to the inherited conservatism of offline RL. Prior works [18, 28, 45, 25, 42, 36, 47] proposed easing conservatism, yet these methods still retain it, limiting sample efficiency. Recently, several works [32, 35, 6, 9] have sought to leverage the multimodality of expressive generative models [24, 1, 11, 37] as policies, but since they rely on single-step prediction, they tend to produce redundant or inconsistent behaviors, resulting in limited sample efficiency. A recently proposed action chunking-based method [22] achieves strong performance through consistent behavior and value learning benefits, yet remains suboptimal due to its fixed chunk length. Our method employs adaptive action chunking to select the optimal chunk length for each state, enabling sample-efficient learning within a limited online budget.

Action Chunking for RL Action chunking has become a key technique in imitation learning by addressing compounding errors and dataset non-stationarity arising from single-step prediction [48, 7, 3, 4]. This technique has been extended to reinforcement learning: in offline RL, various studies [31, 22, 21, 38, 13, 34] have leveraged action chunking, as the resulting horizon reduction facilitates value learning, particularly for long-horizon tasks, while in online RL, prior works [20, 41, 27] have proposed Transformer-based value functions using action chunks to mitigate bias from n -step returns. However, these approaches suffer from limited sample efficiency due to fixed chunk length. Although a prior work [23] in the imitation domain introduces adaptive chunk length selection, it targets only inference time adjustment without incorporating adaptation into training. In contrast, our method leverages action chunking while adaptively adjusting its length during both training and inference, leading to strong sample efficiency in Off2On RL.

7 Conclusion

In this paper, we introduced ACH, a novel offline-to-online RL algorithm that dynamically modulates chunk length during both training and inference. To determine optimal chunk length for a current state, we adopt chunk value as an optimality criterion and employ a causal Transformer-based critic to simultaneously evaluate all candidate chunk lengths in a single forward pass. To enhance exploration during online fine-tuning, we developed a soft policy that adaptively samples chunk lengths based on these values. Our evaluation across 34 challenging tasks demonstrates that ACH consistently outperforms fixed-length baselines, exhibiting superior generalization and learning efficiency in complex environments. We believe that our work opens a new avenue for leveraging adaptive action chunking in reinforcement learning, and that this direction will play an important role in advancing efficient learning in sequential decision making.

References

- [1] Michael Albergo and Eric Vanden-Eijnden. 2023. Building Normalizing Flows with Stochastic Interpolants. In *The Eleventh International Conference on Learning Representations*.
- [2] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. 2023. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*. PMLR, 1577–1594.
- [3] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. 2025. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734* (2025).
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. 2024. π_0 : A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164* (2024).
- [5] Kevin Black, Manuel Y Galliker, and Sergey Levine. [n. d.]. Real-time execution of action chunking flow policies, 2025. URL <https://arxiv.org/abs/2506.07339> ([n. d.]).
- [6] Jongseong Chae, Jongeui Park, Yongjae Shin, Gyeongmin Kim, Seungyul Han, and Youngchul Sung. [n. d.]. Flow Actor-Critic for Offline Reinforcement Learning. In *The Fourteenth International Conference on Learning Representations*.
- [7] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2025. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research* 44, 10-11 (2025), 1684–1704.
- [8] Perry Dong, Qiyang Li, Dorsa Sadigh, and Chelsea Finn. 2025. Expo: Stable reinforcement learning with expressive policies. *arXiv preprint arXiv:2507.07986* (2025).
- [9] Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. 2023. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573* (2023).
- [10] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415* (2016).
- [11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [12] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. 2025. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054* (2025).
- [13] Changyeon Kim, Haeone Lee, Younggyo Seo, Kimin Lee, and Yuke Zhu. 2026. DEAS: DETached value learning with Action Sequence for Scalable Offline RL. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=bVTaAXeBmE>
- [14] Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645* (2025).
- [15] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*.
- [16] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. 2022. Offline Reinforcement Learning with Implicit Q-Learning. In *International Conference on Learning Representations*.
- [17] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. 2020. Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems* 33 (2020), 1179–1191.

- [18] Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. 2022. Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on Robot Learning*. PMLR, 1702–1712.
- [19] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643* (2020).
- [20] Ge Li, Dong Tian, Hongyi Zhou, Xinkai Jiang, Rudolf Lioutikov, and Gerhard Neumann. 2025. TOP-ERL: Transformer-based Off-Policy Episodic Reinforcement Learning. In *The Thirteenth International Conference on Learning Representations*.
- [21] Qiyang Li, Seohong Park, and Sergey Levine. 2026. Decoupled Q-Chunking. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=aqGNdZQL91>
- [22] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. 2025. Reinforcement Learning with Action Chunking. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [23] Yuanchang Liang, Shikai Li, Shiji Song, and Gao Huang. 2026. Adaptive Action Chunking at Inference-time for Vision-Language-Action Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv preprint arXiv:2604.04161.
- [24] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. 2023. Flow Matching for Generative Modeling. In *The Eleventh International Conference on Learning Representations*.
- [25] Qin-Wen Luo, Ming-Kun Xie, Yewen Wang, and Sheng-Jun Huang. 2024. Optimistic critic reconstruction and constrained fine-tuning for general offline-to-online RL. *Advances in Neural Information Processing Systems* 37 (2024), 108167–108207.
- [26] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. 2021. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. In *5th Annual Conference on Robot Learning*.
- [27] CF Nagy, Onur Celik, Emiliyan Gospodinov, Florian Seligmann, Weiran Liao, Aryan Kaushik, and Gerhard Neumann. 2026. SEAR: Sample Efficient Action Chunking Reinforcement Learning. *arXiv preprint arXiv:2603.01891* (2026).
- [28] Mitsuhiro Nakamoto, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. 2023. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems* 36 (2023), 62244–62269.
- [29] Kwanyoung Park, Seohong Park, Youngwoon Lee, and Sergey Levine. 2026. Scalable Offline Model-Based RL with Action Chunks. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=WxGb9unEHo>
- [30] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. 2025. OGBench: Benchmarking Offline Goal-Conditioned RL. In *The Thirteenth International Conference on Learning Representations*.
- [31] Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. 2025. Horizon Reduction Makes RL Scalable. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [32] Seohong Park, Qiyang Li, and Sergey Levine. 2025. Flow Q-Learning. In *International Conference on Machine Learning (ICML)*.
- [33] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. 2025. Leave no observation behind: Real-time correction for vla action chunks. *arXiv preprint arXiv:2509.23224* (2025).

- [34] Younggyo Seo and Pieter Abbeel. 2025. Coarse-to-fine Q-Network with Action Sequence for Data-Efficient Reinforcement Learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=VoFXUNc9Zh>
- [35] Yongjae Shin, Jongseong Chae, Jongeui Park, and Youngchul Sung. 2026. Flow Matching with Injected Noise for Offline-to-Online Reinforcement Learning. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=6wd38R8LOZ>
- [36] Yongjae Shin, Jeonghye Kim, Whiyoung Jung, Sunghoon Hong, Deunsol Yoon, Youngsoo Jang, Geon-Hyeong Kim, Jongseong Chae, Youngchul Sung, Kanghoon Lee, and Woohyung Lim. 2025. Online Pre-Training for Offline-to-Online Reinforcement Learning. In *Forty-second International Conference on Machine Learning*.
- [37] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*. pmlr, 2256–2265.
- [38] Gwanwoo Song, Kwanyoung Park, and Youngwoon Lee. 2026. Chunk-Guided Q-Learning. *arXiv preprint arXiv:2603.13971* (2026).
- [39] Richard S Sutton, Andrew G Barto, et al. 1998. *Reinforcement learning: An introduction*. Vol. 1. MIT press Cambridge.
- [40] Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. 2023. Revisiting the minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 11592–11620.
- [41] Dong Tian, Onur Celik, and Gerhard Neumann. 2026. Chunking the Critic: A Transformer-based Soft Actor-Critic with N-Step Returns. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=rb5eTktqbc>
- [42] Shenzhi Wang, Qisen Yang, Jiawei Gao, Matthieu Lin, Hao Chen, Liwei Wu, Ning Jia, Shiji Song, and Gao Huang. 2023. Train once, get a family: State-adaptive balances for offline-to-online reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 47081–47104.
- [43] Jialong Wu, Haixu Wu, Zihan Qiu, Jianmin Wang, and Mingsheng Long. 2022. Supported policy optimization for offline reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 31278–31291.
- [44] Jiarui Yang, Bin Zhu, Jingjing Chen, and Yu-Gang Jiang. 2026. Actor-critic for continuous action chunks: A reinforcement learning framework for long-horizon robotic manipulation with sparse reward. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 18692–18700.
- [45] Zishun Yu and Xinhua Zhang. 2023. Actor-critic alignment for offline-to-online reinforcement learning. In *International Conference on Machine Learning*. PMLR, 40452–40474.
- [46] Haichao Zhang, Wei Xu, and Haonan Yu. 2023. Policy Expansion for Bridging Offline-to-Online Reinforcement Learning. In *The Eleventh International Conference on Learning Representations*.
- [47] Yinmin Zhang, Jie Liu, Chuming Li, Yazhe Niu, Yaodong Yang, Yu Liu, and Wanli Ouyang. 2024. A Perspective of Q-value Estimation on Offline-to-Online Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 16908–16916.
- [48] Tony Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. *Robotics: Science and Systems XIX* (2023).
- [49] Zhiyuan Zhou, Andy Peng, Qiyang Li, Sergey Levine, and Aviral Kumar. 2024. Efficient Online Reinforcement Learning Fine-Tuning Need Not Retain Offline Data. In *The Thirteenth International Conference on Learning Representations*.

A Limitations

Since the proposed method employs a Transformer as the value function, which is relatively heavier than the MLP used in prior methods, training requires comparatively more time. We report this in Appendix C.2. A potential direction for future work is to identify a lighter alternative to the Transformer that still enables multiple chunk value estimation. Additionally, as with existing methods, the proposed method executes the sample action chunk in an open-loop manner, which may limit its applicability in environments with rapidly changing dynamics. In such cases, replanning during action chunk execution [5, 33] would be necessary, and this constitutes another promising direction for future research.

B Experimental Details

In Section 5, we validate the efficacy of the proposed method through comprehensive evaluations across 34 distinct tasks. This section provides the specific details of the experiments used to obtain these results.

B.1 Benchmarks and Tasks

We conduct experiments on 34 tasks from OGBench [30] and 4 tasks from robomimic [26].

Following standard protocols in offline-to-online RL[32, 35], we utilize the single-task variant across six environments: `antmaze-giant`, `antsoccer-arena`, `cube-quadruple`, `cube-triple`, `humanoidmaze-medium`, `puzzle-4x4`. Each environment includes five distinct tasks.

- `Antmaze` and `Antsoccer`: The objective is to control a robot with an 8-dimensional action space to reach a designated goal.
- `Cube`: A robotic arm with a 5-dimensional action space performs various pick-and-place tasks.
- `Humanoidmaze`: This task involves navigating a humanoid robot in a 21-dimensional action space to a goal.
- `Puzzle`: A robotic arm with a 5-dimensional action space, press buttons in a specific manner to solve the puzzle.

For the datasets, we employ the navigate dataset for `antmaze`, `antsoccer`, and `humanoidmaze`, which consist of 1M, 1M, and 2M transitions, respectively. The play dataset is utilized for `cube` and `puzzle`, containing 3M transitions for `cube-triple`, 10M transitions for `cube-quadruple`, and 1M transitions for `puzzle`.

We further evaluate the method on four specific tasks within the robomimic benchmark. All experiments in this domain utilize the multi-human dataset, which consists of 300 successful trajectories for each task.

- `lift`: A single robotic arm with a 7-dimensional action space lifts a small cube.
- `can`: A single robotic arm with a 7-dimensional action space picks up a can and places it into a target bin.
- `square`: A single robotic arm with a 7-dimensional action space places a square nut onto a rod.
- `transport`: Two robotic arms with a combined 14-dimensional action space perform a collaborative maneuver. One arm lifts a hammer and passes it to the other arm, which then places the object into a target bin.

B.2 Evaluation

We evaluate all baselines every 100K environment steps. Each evaluation measures the success rate over 50 episodes.

B.3 Baselines

For the comparative analysis, we implement the baselines by leveraging the experimental settings of FINO [35] and QC [22]. Specifically, the implementations of RLPD, FQL, and FINO are based on the paper and official code of FINO¹, while the BFN and QC follow the paper and official code of QC². In accordance with previous studies [32, 35], RLPD is configured with an update-to-data ratio of 1 and two value functions. The number of sampled actions for BFN is fixed at 4. The BC coefficient α for FQL, FINO, and QC are summarized in Table 3, and the common hyperparameters utilized across all baselines are presented in Table 2.

Table 2: Shared Hyperparameters

Hyperparameter	Value
Learning rate	0.0003
Optimizer	Adam [15]
Batch size	256
Hidden dimensions	[512, 512, 512, 512]
Nonlinearity	GELU [10]
Target network update rate	0.005
Discount factor γ	0.99 (default), 0.995 (antmaze, humanoidmaze, antsoccer)
Flow steps	10

Table 3: Task-specific BC coefficient α for each baseline.

Task	FQL	FINO	QC	ACH
antmaze-giant-navigate	10	10	30	30
antsoccer-arena-navigate	30	30	30	30
cube-quadruple-play-10M	300	300	100	300
cube-triple-play	300	300	100	300
humanoidmaze-medium-navigate	100	100	100	100
puzzle-4x4-play	1000	1000	300	300
lift	10000	10000	10000	10000
can	10000	10000	10000	10000
square	10000	10000	10000	10000
transport	10000	10000	10000	1000

B.4 ACH

The proposed method, ACH, is built upon QC as the backbone model and incorporates a transformer architecture into the value function. This transformer configuration consists of an embedding dimension of 128, 4 heads, and 2 attention layers. As with other baselines, ACH utilizes environment-specific BC coefficient α , which is provided in Table 3. We set the chunk length to 10 for all environments; however, for humanoidmaze, where the action dimension is 21 and relatively large, learning with a chunk length of 10 becomes difficult for the policy, so we reduce the chunk length to 5 in this case only. For adaptive chunk length sampling at inference, the action chunk with the highest action-value is selected to ensure consistent evaluation, thereby rendering the policy deterministic. The pseudo-code for ACH is presented in Algorithm 1.

¹<https://github.com/CTID282/FINO>

²<https://github.com/ColinQiyangLi/qc>

Algorithm 1 ACH: Adaptive action CHunking for offline-to-online rl

```
1: Inputs: offline dataset  $D$ , behavior policy  $\pi_\theta$ , one-step policy  $\pi_\omega$ , transformer-based value  
   function  $Q_\phi$ , horizon length  $h$ , action chunk buffer  $\mathcal{B} \leftarrow \emptyset$   
2: while in offline learning do  
3:   Update  $\theta, \omega, \phi$  based on Equations 1, 9, 8.  
4: end while  
5: while in online learning do  
6:   if  $\mathcal{B}$  is empty then  
7:     Sample full-length action chunk  $\mathbf{a}_{t:t+h} \sim \pi_\omega(s_t)$   
8:     Compute values  $\{Q_\phi(s_t, \mathbf{a}_t), \dots, Q_\phi(s_t, \mathbf{a}_{t+h})\}$   
9:     Calculate sampling probability  $p(\ell)$  from Equation 10  
10:    Select length  $\ell \sim p(\ell)$   
11:    Store  $\{\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+\ell}\}$  in  $\mathcal{B}$   
12:   end if  
13:   Get  $\mathbf{a}_t$  from  $\mathcal{B}$  and execute  $\mathbf{a}_t$   
14:   Update  $\theta, \omega, \phi$  based on Equations 1, 9, 8.  
15: end while
```

C Additional Experiments

C.1 Comparison with Fixed-Length Baselines

Table 4: Aggregated results of ACH and baselines on OGBench. All methods are evaluated after 1M steps of offline learning followed by 1M steps of online learning. We report the mean and 95% confidence intervals across 10 seeds. Results for each environment are averaged over five tasks.

Task	H1 (FQL)	H3	H5 (QC)	H10	ACH
antmaze-giant-navigate	10 $\pm$ 5 $\rightarrow$ 74 $\pm$ 7	0 $\pm$ 0 $\rightarrow$ 71 $\pm$ 7	0 $\pm$ 0 $\rightarrow$ 47 $\pm$ 8	0 $\pm$ 0 $\rightarrow$ 20 $\pm$ 6	1 $\pm$ 1 $\rightarrow$ 72 $\pm$ 2
antsoccer-arena-navigate	35 $\pm$ 2 $\rightarrow$ 70 $\pm$ 5	50 $\pm$ 3 $\rightarrow$ 82 $\pm$ 2	10 $\pm$ 2 $\rightarrow$ 67 $\pm$ 2	0 $\pm$ 0 $\rightarrow$ 12 $\pm$ 2	0 $\pm$ 0 $\rightarrow$ 84 $\pm$ 1
cube-quadruple-play-10M	0 $\pm$ 0 $\rightarrow$ 2 $\pm$ 1	1 $\pm$ 0 $\rightarrow$ 8 $\pm$ 5	1 $\pm$ 1 $\rightarrow$ 33 $\pm$ 3	1 $\pm$ 0 $\rightarrow$ 41 $\pm$ 3	0 $\pm$ 0 $\rightarrow$ 52 $\pm$ 6
cube-triple-play	2 $\pm$ 1 $\rightarrow$ 19 $\pm$ 2	5 $\pm$ 1 $\rightarrow$ 35 $\pm$ 3	4 $\pm$ 2 $\rightarrow$ 57 $\pm$ 4	3 $\pm$ 1 $\rightarrow$ 70 $\pm$ 4	1 $\pm$ 1 $\rightarrow$ 67 $\pm$ 3
humanoidmaze-medium-navigate	36 $\pm$ 3 $\rightarrow$ 56 $\pm$ 3	53 $\pm$ 3 $\rightarrow$ 61 $\pm$ 7	15 $\pm$ 2 $\rightarrow$ 32 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	45 $\pm$ 2 $\rightarrow$ 82 $\pm$ 1
puzzle-4x4-play	14 $\pm$ 2 $\rightarrow$ 54 $\pm$ 6	20 $\pm$ 3 $\rightarrow$ 50 $\pm$ 8	11 $\pm$ 2 $\rightarrow$ 38 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 20 $\pm$ 5	13 $\pm$ 3 $\rightarrow$ 90 $\pm$ 8

In Section 3, we state the motivation of our approach that the optimal chunk length is inherently state-dependent and varies across different tasks. To empirically validate the impact of this phenomenon, we evaluate the performance of fixed-length baselines using four distinct values: 1, 3, 5, and 10. As summarized in Table 4, the results confirm that the performance of these baselines fluctuates significantly across environments depending on the fixed length. Although certain fixed lengths result in comparable performance to the proposed method in some environments, they frequently show degradation in other environments. This discrepancy suggests that a fixed chunk length only yields high performance when it happens to coincide with the average of the optimal lengths required for a particular environment. In contrast, the proposed method consistently demonstrates superior performance across all environments. This outcome indicates that our approach effectively identifies and adapts to the optimal chunk length for each state, ultimately facilitating highly efficient and robust learning.

C.2 Computational Cost

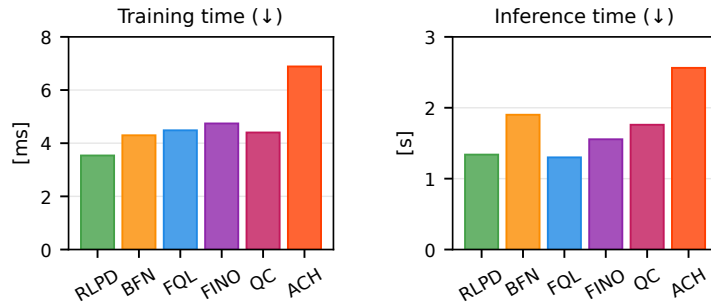


Figure 8: Comparison of computational costs between the proposed method and baselines on the antmaze-giant-navigate task. Training time represents the duration per step during the online learning, and inference time denotes the time required for a single episode rollout. All measurements are conducted on a single NVIDIA RTX 3090 GPU.

In Figure 8, we provide the computational costs of the proposed method and the baselines during both the training and inference phases. Since ACH utilizes a relatively heavy transformer architecture and requires value estimation across all chunk lengths during inference, the computational time for ACH is higher than that of other baselines. However, the increase is not prohibitive, as it remains 1.3 to 1.4 times the processing time of the baselines. Considering the significant performance gains achieved by our method, this additional overhead is well within a justifiable limit.

D Complete Results

D.1 Main Experiments

We present the per-task results corresponding to Table 1 in Table 5, and the corresponding learning curves in Figure 10. For the ablation study shown in Figure 6, we report the results across all tasks in Table 6.

D.2 Behavior Analysis

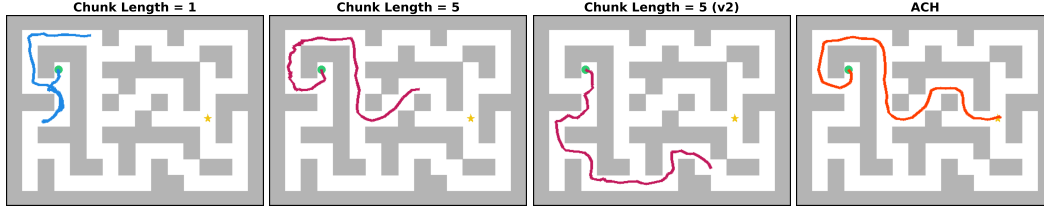


Figure 9: Behavioral comparison of ACH and fixed-length baselines in the antmaze-giant environment. Starting from a shared initial state, all algorithms execute the same number of total action steps. Agent trajectories are constructed by connecting sequential positions at each timestep. The initial state and goal state are marked by a green circle and a yellow star, respectively.

In Figure 5, we validate the efficacy of the proposed method by analyzing the behaviors required to solve tasks in a practical environment, comparing our approach with fixed-length baselines. In this section, Figure 9 presents the full layout of the environment and an additional behavioral pattern observed when the chunk length is fixed at 5 (v2). The trajectory for this baseline reveals a significant detour instead of following the shortest path to the goal, which eventually leads to a failure to reach the goal within the time limit. This outcome stems from the rigid nature of the fixed-length constraint. If the agent deviates from the optimal path during the early stages, the lack of flexibility prevents it from recovering and forces it to follow a suboptimal route. These findings highlight the inherent limitation of conventional methods that rely on a static chunking strategy.

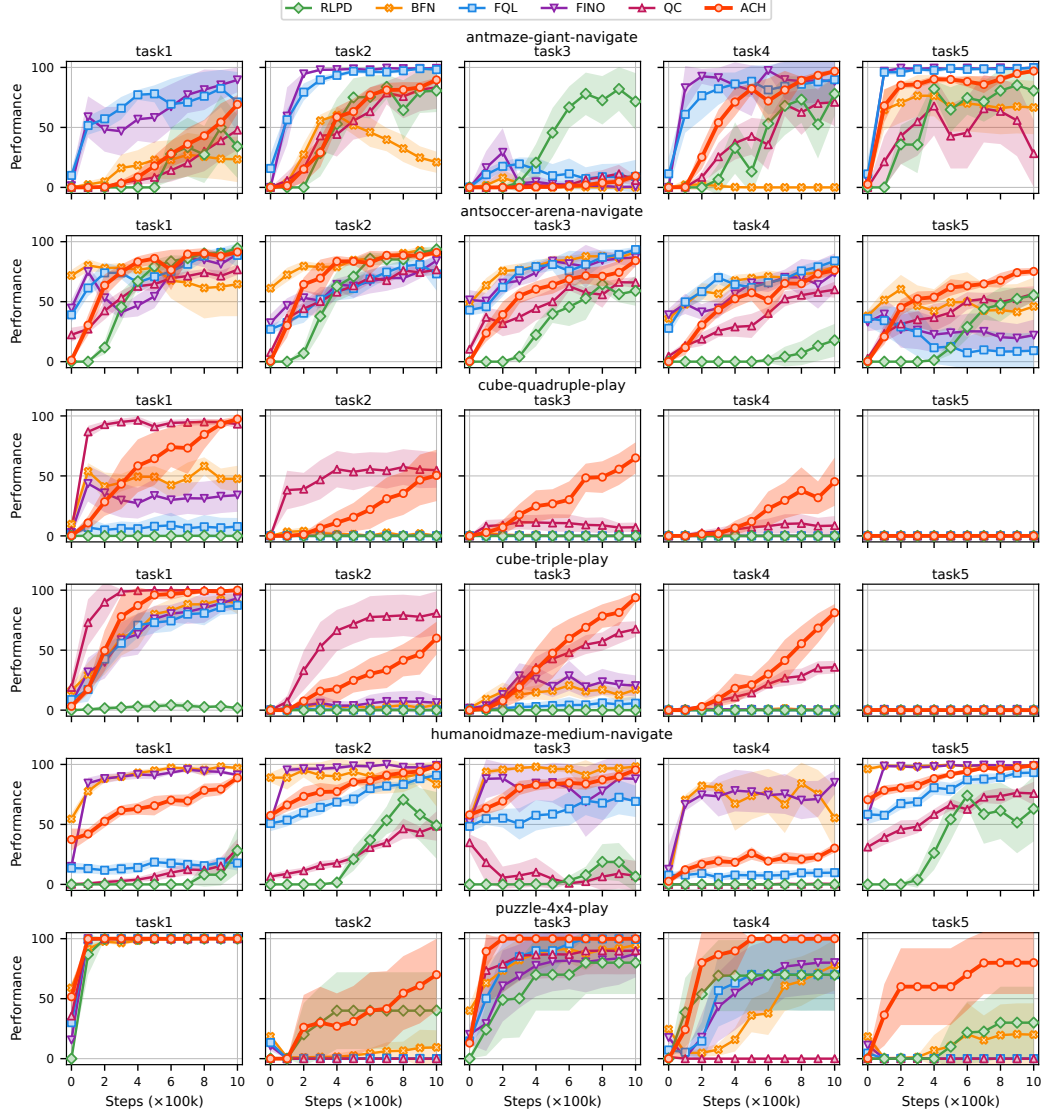


Figure 10: Learning curves during online learning for all tasks in OGBench. Solid lines represent the mean across 10 seeds, and shaded regions denote the 95% confidence interval.

Table 5: Complete results for Table 1 and Figure 4, showing the performance immediately after 1M steps of offline learning and 1M steps of online learning. Results represent the mean $\pm$ 95% confidence interval calculated across 10 seeds.

Environment	RLPD	BFN	FQL	FINO	QC	ACH
OGBench antmaze-giant-navigate-singletask-task1	0±0 → 34±26	0±0 → 23±19	10±6 → 71±25	1±9 → 90 ±10	0±0 → 48±17	0±0 → 69±8
OGBench antmaze-giant-navigate-singletask-task2	0±0 → 81±18	0±0 → 21±9	16±12 → 98 ±1	0±0 → 99 ±1	0±0 → 83±18	0±0 → 90±2
OGBench antmaze-giant-navigate-singletask-task3	0±0 → 72 ±24	0±0 → 0±0	0±0 → 9±14	0±0 → 1±1	0±0 → 6±4	0±0 → 10±3
OGBench antmaze-giant-navigate-singletask-task4	0±0 → 78±18	0±0 → 0±0	11±14 → 90±16	0±0 → 88±19	0±0 → 71±20	0±0 → 97 ±1
OGBench antmaze-giant-navigate-singletask-task5	0±0 → 81±19	2±2 → 67±22	11±13 → 100 ±0	7±8 → 100 ±0	0±0 → 28±28	3±5 → 97 ±1
OGBench antsoccer-arena-navigate-singletask-task1	0±0 → 95 ±2	72±5 → 65±27	39±6 → 89±10	45±7 → 90 ±4	22±6 → 77±4	1±1 → 91 ±2
OGBench antsoccer-arena-navigate-singletask-task2	0±0 → 94 ±2	61±4 → 91 ±2	27±4 → 73±14	32±7 → 84±6	8±2 → 77±4	0±1 → 91 ±2
OGBench antsoccer-arena-navigate-singletask-task3	0±0 → 59±12	48±3 → 91 ±3	43±5 → 93 ±2	51±8 → 87±9	10±3 → 66±5	0±0 → 84±5
OGBench antsoccer-arena-navigate-singletask-task4	0±0 → 18±14	36±6 → 79 ±5	28±6 → 84 ±4	39±5 → 74±16	5±2 → 60±7	0±0 → 76±3
OGBench antsoccer-arena-navigate-singletask-task5	0±0 → 55±6	38±5 → 46±24	36±7 → 9±15	33±8 → 22±13	3±2 → 57±6	0±0 → 75 ±3
OGBench cube-quadruple-play-10M-singletask-task1	0±0 → 0±0	10±5 → 48±11	1±1 → 8±7	3±2 → 34±14	4±3 → 93 ±3	0±0 → 97 ±2
OGBench cube-quadruple-play-10M-singletask-task2	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 55 ±15	0±0 → 50±21
OGBench cube-quadruple-play-10M-singletask-task3	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 7±4	0±0 → 65 ±13
OGBench cube-quadruple-play-10M-singletask-task4	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 9±8	0±0 → 45 ±20
OGBench cube-quadruple-play-10M-singletask-task5	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0
OGBench cube-triple-play-singletask-task1	0±0 → 2±1	16±5 → 89±9	9±4 → 87±7	6±3 → 93±9	19±8 → 100 ±1	3±3 → 100 ±0
OGBench cube-triple-play-singletask-task2	0±0 → 0±0	0±1 → 5±4	0±0 → 0±0	0±0 → 6±5	0±0 → 81 ±18	0±0 → 60±14
OGBench cube-triple-play-singletask-task3	0±0 → 0±0	2±2 → 17±8	0±0 → 6±6	2±2 → 20±9	0±0 → 68±7	0±0 → 94 ±4
OGBench cube-triple-play-singletask-task4	0±0 → 0±0	0±0 → 0±1	0±0 → 0±0	0±0 → 0±0	0±0 → 36±5	0±0 → 81 ±7
OGBench cube-triple-play-singletask-task5	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0	0±0 → 0±0
OGBench humanoidmaze-medium-navigate-singletask-task1	0±0 → 28±19	55±5 → 97 ±2	14±5 → 18±6	15±4 → 91±1	0±0 → 30±8	37±9 → 89±3
OGBench humanoidmaze-medium-navigate-singletask-task2	0±0 → 49±28	89±4 → 84±19	51±6 → 91±2	52±2 → 99 ±1	7±2 → 49±12	57±4 → 99 ±1
OGBench humanoidmaze-medium-navigate-singletask-task3	0±0 → 7±13	54±11 → 98 ±2	48±5 → 69±13	52±12 → 88±18	35±7 → 7±12	58±4 → 95 ±2
OGBench humanoidmaze-medium-navigate-singletask-task4	0±0 → 0±0	4±7 → 55±26	8±3 → 10±4	13±20 → 85 ±10	0±0 → 0±1	3±2 → 30±4
OGBench humanoidmaze-medium-navigate-singletask-task5	0±0 → 63±27	96±3 → 99 ±1	58±8 → 93±4	57±1 → 99 ±1	31±5 → 76±10	71±5 → 99 ±1
OGBench puzzle-4x4-play-singletask-task1	0±0 → 100 ±0	59±4 → 100 ±0	30±6 → 100 ±0	16±9 → 100 ±0	36±8 → 100 ±0	52±11 → 100 ±0
OGBench puzzle-4x4-play-singletask-task2	0±0 → 40±32	19±2 → 9±14	13±3 → 0±0	11±4 → 0±0	0±0 → 0±0	0±0 → 70 ±30
OGBench puzzle-4x4-play-singletask-task3	0±0 → 80±26	40±5 → 95±5	15±3 → 99 ±1	20±8 → 87±20	16±4 → 90±20	13±7 → 100 ±0
OGBench puzzle-4x4-play-singletask-task4	0±0 → 70±30	25±3 → 78±21	7±2 → 70±30	18±5 → 80±26	1±1 → 0±0	0±0 → 100 ±0
OGBench puzzle-4x4-play-singletask-task5	0±0 → 30±30	18±5 → 20±26	5±2 → 0±0	11±6 → 0±0	0±0 → 0±0	0±0 → 80 ±26
robomimic lift-mh	0±0 → 100 ±1	90±3 → 98 ±1	83±5 → 94±3	87±3 → 99 ±1	97±2 → 100 ±0	96±1 → 100 ±0
robomimic can-mh	0±0 → 70±5	64±6 → 80±4	39±7 → 71±6	53±9 → 88±5	89±3 → 93±2	88±2 → 99 ±1
robomimic square-mh	0±0 → 72±3	14±2 → 28±3	4±2 → 13±3	8±3 → 29±4	35±5 → 81±3	42±5 → 94 ±2
robomimic transport-mh	0±0 → 0±0	1±1 → 1±1	0±0 → 0±0	0±0 → 0±0	13±4 → 22±6	0±0 → 39 ±4

Table 6: Complete results for Figure 6, showing the performance immediately after 1M steps of offline learning and 1M steps of online learning. Results represent the mean $\pm$ 95% confidence interval calculated across 10 seeds.

Environment	Random	Uniform	Greedy	ACH
OGBench antmaze-giant-navigate-singletask-task1	0 $\pm$ 0 $\rightarrow$ 25 $\pm$ 18	0 $\pm$ 0 $\rightarrow$ 26 $\pm$ 8	0 $\pm$ 0 $\rightarrow$ 77 $\pm$ 7	0 $\pm$ 0 $\rightarrow$ 69 $\pm$ 8
OGBench antmaze-giant-navigate-singletask-task2	0 $\pm$ 0 $\rightarrow$ 68 $\pm$ 18	0 $\pm$ 0 $\rightarrow$ 88 $\pm$ 2	0 $\pm$ 0 $\rightarrow$ 86 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 90 $\pm$ 2
OGBench antmaze-giant-navigate-singletask-task3	0 $\pm$ 0 $\rightarrow$ 13 $\pm$ 8	0 $\pm$ 0 $\rightarrow$ 2 $\pm$ 1	0 $\pm$ 0 $\rightarrow$ 40 $\pm$ 7	0 $\pm$ 0 $\rightarrow$ 10 $\pm$ 3
OGBench antmaze-giant-navigate-singletask-task4	0 $\pm$ 0 $\rightarrow$ 45 $\pm$ 25	0 $\pm$ 0 $\rightarrow$ 91 $\pm$ 3	0 $\pm$ 0 $\rightarrow$ 92 $\pm$ 3	0 $\pm$ 0 $\rightarrow$ 97 $\pm$ 1
OGBench antmaze-giant-navigate-singletask-task5	0 $\pm$ 0 $\rightarrow$ 27 $\pm$ 26	4 $\pm$ 3 $\rightarrow$ 91 $\pm$ 4	6 $\pm$ 5 $\rightarrow$ 94 $\pm$ 3	3 $\pm$ 5 $\rightarrow$ 97 $\pm$ 1
OGBench antsoccer-arena-navigate-singletask-task1	23 $\pm$ 6 $\rightarrow$ 70 $\pm$ 6	1 $\pm$ 1 $\rightarrow$ 83 $\pm$ 4	1 $\pm$ 1 $\rightarrow$ 89 $\pm$ 3	1 $\pm$ 1 $\rightarrow$ 91 $\pm$ 2
OGBench antsoccer-arena-navigate-singletask-task2	7 $\pm$ 2 $\rightarrow$ 73 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 80 $\pm$ 3	0 $\pm$ 0 $\rightarrow$ 90 $\pm$ 3	0 $\pm$ 1 $\rightarrow$ 91 $\pm$ 2
OGBench antsoccer-arena-navigate-singletask-task3	1 $\pm$ 1 $\rightarrow$ 23 $\pm$ 7	0 $\pm$ 0 $\rightarrow$ 73 $\pm$ 4	0 $\pm$ 1 $\rightarrow$ 78 $\pm$ 10	0 $\pm$ 0 $\rightarrow$ 84 $\pm$ 5
OGBench antsoccer-arena-navigate-singletask-task4	6 $\pm$ 2 $\rightarrow$ 55 $\pm$ 7	0 $\pm$ 0 $\rightarrow$ 63 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 75 $\pm$ 3	0 $\pm$ 0 $\rightarrow$ 76 $\pm$ 3
OGBench antsoccer-arena-navigate-singletask-task5	4 $\pm$ 2 $\rightarrow$ 55 $\pm$ 6	0 $\pm$ 0 $\rightarrow$ 67 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 72 $\pm$ 6	0 $\pm$ 0 $\rightarrow$ 75 $\pm$ 3
OGBench cube-quadruple-play-10M-singletask-task1	2 $\pm$ 1 $\rightarrow$ 95 $\pm$ 3	0 $\pm$ 0 $\rightarrow$ 13 $\pm$ 20	0 $\pm$ 0 $\rightarrow$ 36 $\pm$ 29	0 $\pm$ 0 $\rightarrow$ 97 $\pm$ 2
OGBench cube-quadruple-play-10M-singletask-task2	0 $\pm$ 0 $\rightarrow$ 55 $\pm$ 14	0 $\pm$ 0 $\rightarrow$ 1 $\pm$ 2	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 50 $\pm$ 21
OGBench cube-quadruple-play-10M-singletask-task3	0 $\pm$ 0 $\rightarrow$ 7 $\pm$ 4	0 $\pm$ 0 $\rightarrow$ 31 $\pm$ 20	0 $\pm$ 0 $\rightarrow$ 27 $\pm$ 16	0 $\pm$ 0 $\rightarrow$ 65 $\pm$ 13
OGBench cube-quadruple-play-10M-singletask-task4	0 $\pm$ 0 $\rightarrow$ 6 $\pm$ 5	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 14 $\pm$ 17	0 $\pm$ 0 $\rightarrow$ 45 $\pm$ 20
OGBench cube-quadruple-play-10M-singletask-task5	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0
OGBench cube-triple-play-singletask-task1	14 $\pm$ 6 $\rightarrow$ 100 $\pm$ 0	4 $\pm$ 4 $\rightarrow$ 7 $\pm$ 8	2 $\pm$ 2 $\rightarrow$ 92 $\pm$ 12	3 $\pm$ 3 $\rightarrow$ 100 $\pm$ 0
OGBench cube-triple-play-singletask-task2	0 $\pm$ 0 $\rightarrow$ 75 $\pm$ 17	0 $\pm$ 0 $\rightarrow$ 15 $\pm$ 19	0 $\pm$ 0 $\rightarrow$ 44 $\pm$ 29	0 $\pm$ 0 $\rightarrow$ 60 $\pm$ 14
OGBench cube-triple-play-singletask-task3	1 $\pm$ 1 $\rightarrow$ 52 $\pm$ 11	1 $\pm$ 1 $\rightarrow$ 80 $\pm$ 19	0 $\pm$ 0 $\rightarrow$ 81 $\pm$ 13	0 $\pm$ 0 $\rightarrow$ 94 $\pm$ 4
OGBench cube-triple-play-singletask-task4	0 $\pm$ 0 $\rightarrow$ 27 $\pm$ 10	0 $\pm$ 0 $\rightarrow$ 20 $\pm$ 25	0 $\pm$ 0 $\rightarrow$ 50 $\pm$ 18	0 $\pm$ 0 $\rightarrow$ 81 $\pm$ 7
OGBench cube-triple-play-singletask-task5	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0
OGBench humanoidmaze-medium-navigate-singletask-task1	0 $\pm$ 0 $\rightarrow$ 30 $\pm$ 12	49 $\pm$ 5 $\rightarrow$ 31 $\pm$ 2	38 $\pm$ 5 $\rightarrow$ 88 $\pm$ 6	37 $\pm$ 9 $\rightarrow$ 89 $\pm$ 3
OGBench humanoidmaze-medium-navigate-singletask-task2	8 $\pm$ 2 $\rightarrow$ 54 $\pm$ 5	52 $\pm$ 4 $\rightarrow$ 78 $\pm$ 7	63 $\pm$ 3 $\rightarrow$ 94 $\pm$ 3	57 $\pm$ 4 $\rightarrow$ 99 $\pm$ 1
OGBench humanoidmaze-medium-navigate-singletask-task3	23 $\pm$ 10 $\rightarrow$ 9 $\pm$ 16	42 $\pm$ 7 $\rightarrow$ 50 $\pm$ 5	51 $\pm$ 5 $\rightarrow$ 89 $\pm$ 4	58 $\pm$ 4 $\rightarrow$ 95 $\pm$ 2
OGBench humanoidmaze-medium-navigate-singletask-task4	0 $\pm$ 0 $\rightarrow$ 1 $\pm$ 1	6 $\pm$ 3 $\rightarrow$ 5 $\pm$ 2	3 $\pm$ 4 $\rightarrow$ 22 $\pm$ 4	3 $\pm$ 2 $\rightarrow$ 30 $\pm$ 4
OGBench humanoidmaze-medium-navigate-singletask-task5	29 $\pm$ 6 $\rightarrow$ 85 $\pm$ 6	67 $\pm$ 8 $\rightarrow$ 82 $\pm$ 7	70 $\pm$ 4 $\rightarrow$ 96 $\pm$ 1	71 $\pm$ 5 $\rightarrow$ 99 $\pm$ 1
OGBench puzzle-4x4-play-singletask-task1	35 $\pm$ 7 $\rightarrow$ 99 $\pm$ 1	63 $\pm$ 15 $\rightarrow$ 100 $\pm$ 0	49 $\pm$ 6 $\rightarrow$ 100 $\pm$ 0	52 $\pm$ 11 $\rightarrow$ 100 $\pm$ 0
OGBench puzzle-4x4-play-singletask-task2	0 $\pm$ 0 $\rightarrow$ 20 $\pm$ 26	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 20 $\pm$ 26	0 $\pm$ 0 $\rightarrow$ 70 $\pm$ 30
OGBench puzzle-4x4-play-singletask-task3	14 $\pm$ 7 $\rightarrow$ 80 $\pm$ 26	28 $\pm$ 11 $\rightarrow$ 50 $\pm$ 33	9 $\pm$ 4 $\rightarrow$ 100 $\pm$ 0	13 $\pm$ 7 $\rightarrow$ 100 $\pm$ 0
OGBench puzzle-4x4-play-singletask-task4	0 $\pm$ 0 $\rightarrow$ 10 $\pm$ 20	1 $\pm$ 1 $\rightarrow$ 30 $\pm$ 30	0 $\pm$ 1 $\rightarrow$ 90 $\pm$ 20	0 $\pm$ 0 $\rightarrow$ 100 $\pm$ 0
OGBench puzzle-4x4-play-singletask-task5	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 0 $\pm$ 0	0 $\pm$ 0 $\rightarrow$ 20 $\pm$ 26	0 $\pm$ 0 $\rightarrow$ 80 $\pm$ 26